

GPUMODE Lecture Study Notes: Lecture 90: Building Resilient ML Engineering Skills

Core Problem the Lecture Addresses

This lecture is about building a resilient machine learning engineering career in an environment where models, frameworks, GPU generations, cloud providers, networking stacks, distributed training frameworks, and AI coding tools change constantly.

The central technical and career problem is:

Resilience = ability to keep delivering correct and efficient systems

even when the following variables change:

{GPU generation, CUDA version, PyTorch version, network topology, storage backend, cloud provider, model a

The speaker argues that resilience does not come from memorizing one particular library or benchmark result. It comes from building transferable engineering habits:

- measuring hardware rather than trusting marketing specifications;
- keeping rigorous notes about failures, fixes, versions, and experimental results;
- developing relationships with engineers who understand critical system components;
- understanding bottlenecks across the whole end-to-end stack, not only the GPU kernel;
- separating theoretical peak performance from measurable, workload-specific performance;
- building long-lived technical artifacts such as books, repositories, notes, and benchmarks.

A recurring theme is that an ML workload is not “a GPU job”. It is an ensemble system:

ML system = compute+networking+storage+software stack+orchestration+debugging discipline.

If any component is weak, the whole system may underperform. A cluster with world-class accelerators can still waste money if the storage cannot feed the dataloader, if the network exposes communication latency, if checkpointing is slow, or if one bad GPU throttles the synchronous training loop.

Required Background

Accelerators, Nodes, and Clusters

A modern large-scale training system is usually organized hierarchically:

cluster $\rightarrow$ nodes $\rightarrow$ GPUs $\rightarrow$ SMs $\rightarrow$ warps $\rightarrow$ threads.

A **node** may contain multiple GPUs, for example 8 GPUs connected by NVLink and NVSwitch. A **cluster** contains many nodes connected through inter-node networking such as InfiniBand, RoCE, or vendor-specific interconnects.

The key distinction is:

intra-node communication $\neq$ inter-node communication.

Intra-node communication is usually much faster because GPUs can communicate over local high-bandwidth fabrics. Inter-node communication crosses the cluster network and is often the slowest part of distributed training.

Dense FLOPs, Sparse FLOPs, and Marketing Numbers

Vendors often report accelerator performance in terms of teraFLOPs or petaFLOPs. A floating-point operation count is a way to estimate raw compute throughput:

$$\text{Throughput} = \frac{\text{number of floating-point operations}}{\text{elapsed time}}.$$

For matrix multiplication,

$$C = AB,$$

where

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N},$$

the approximate operation count is

$$\text{FLOPs}_{\text{GEMM}} = 2MKN.$$

The factor 2 comes from one multiply and one add per inner product term. Hardware commonly implements this as a fused multiply-add operation:

$$d = a \cdot b + c.$$

A critical warning from the lecture is that published accelerator FLOP numbers are often not the numbers one can realize in an actual training job. Reasons include:

- sparse FLOP numbers may be advertised instead of dense FLOP numbers;
- boost clocks may be unsustainable under continuous load;
- memory bandwidth may not feed compute units fast enough;
- matrix shapes may be too small or irregular;
- software overheads and synchronization may dominate;
- distributed communication may expose idle time.

MFU and Why It Can Mislead

Model FLOP utilization, commonly abbreviated MFU, measures how much of the theoretical model compute rate is being achieved:

$$\text{MFU} = \frac{\text{measured model FLOPs per second}}{\text{theoretical peak accelerator FLOPs per second}}.$$

The speaker emphasizes that the denominator is often wrong. If the theoretical peak is an unachievable marketing number, then MFU can make a well-optimized training job look mediocre.

A more useful denominator is the maximum performance one can actually measure on the target hardware, with the relevant data type, relevant matrix shapes, and relevant software stack:

$$\text{Realistic MFU} = \frac{\text{measured workload FLOPs per second}}{\text{measured maximum achievable FLOPs per second}}.$$

This transforms optimization from a vague chase toward a fictional 100% into a practical engineering process.

Synchronous Distributed Training

Most large-scale training proceeds synchronously. If one GPU is slow, the entire job can slow down:

$$T_{\text{step}} = \max_{g \in \{1, \dots, G\}} T_{\text{step}}^{(g)}.$$

Here G is the number of GPUs. A single slow or thermally throttled GPU can increase the maximum step time and reduce cluster-wide utilization.

Main Concepts

Resilience Is Built from Notes, Measurement, and People

The lecture begins with a career-level observation: the ML ecosystem changes too quickly for static knowledge to be enough. New models, new libraries, new kernels, new cloud offerings, and new AI coding systems appear continuously. The dependency graph is deep and fragile.

The speaker's primary personal method is rigorous note-taking:

Problem encountered → debugging path → root cause → fix → reusable note.

The value is not only remembering the exact solution. The deeper value is remembering the **process** that led to the solution. Many ML systems failures recur in slightly different forms:

- incompatible package versions;
- hidden environment-variable differences;
- driver or CUDA mismatches;
- networking collectives slower than expected;
- storage too slow for checkpointing;
- one GPU behaving differently from the others;
- changes in PyTorch memory behavior across versions.

The second resilience mechanism is relationships. During the BLOOM 176B training effort, the speaker had to lead training on a scale far larger than his prior experience. The project succeeded partly because he had access to key people from PyTorch, Megatron, DeepSpeed, and the compute site.

The practical lesson is:

In complex ML systems, knowing who can explain a component is often as important as already knowing the component yourself.

The Whole System Matters More Than One GPU Spec

A GPU is only one part of a training system. A simplified training iteration can be modeled as:

$$T_{\text{iteration}} = T_{\text{data}} + T_{\text{compute}} + T_{\text{communication}} + T_{\text{logging}} + T_{\text{misc}}.$$

With overlap, the effective time may be closer to

$$T_{\text{iteration}} = T_{\text{data}} + \max(T_{\text{compute}}, T_{\text{communication}}) + T_{\text{logging}} + T_{\text{misc}},$$

but only if communication can actually be overlapped with compute and data is ready on time.

A cluster is therefore only as strong as its bottleneck:

effective system throughput = $\min(\text{compute throughput}, \text{network throughput}, \text{storage throughput}, \text{software throughput})$

This is why a fast GPU can still idle.

Time to Market and Cost

Two primary business constraints appear throughout the lecture:

Objective = minimize time to useful model subject to budget and reliability constraints.

Time to market matters because model releases can be competitive. Releasing a strong model one day late may lose market attention. Cost matters because accelerator contracts may involve tens of millions of dollars. Wasted hardware time is not a small inefficiency; it can be the difference between a viable project and a failed one.

The economic cost of a run can be approximated as:

$$C_{\text{run}} = G \cdot c_{\text{gpu-hour}} \cdot T_{\text{hours}},$$

where G is the GPU count, $c_{\text{gpu-hour}}$ is cost per GPU-hour, and T_{hours} is wall-clock time.

If utilization is low, the effective cost per useful compute unit rises:

$$C_{\text{effective}} = \frac{C_{\text{run}}}{\text{useful work completed}}.$$

Thus, buying or renting faster GPUs without matching networking and storage may increase nominal hardware quality while worsening effective economics.

Renting Versus Buying

The speaker expresses caution about buying large GPU fleets, especially outside personal development hardware. Hardware ages quickly. Renting can be safer when workloads change and newer accelerators appear frequently.

A practical split is:

local GPU $\rightarrow$ debugging and small experiments,

rented cluster $\rightarrow$ serious training, inference serving, and scaling tests.

This is not a universal rule. The broader principle is that hardware decisions must follow workload requirements.

Cloud Provider Evaluation Requires Defensive Benchmarking

A recurring message is that cloud service provider claims must be validated. Before committing to expensive contracts, the speaker recommends requesting access to representative nodes and running a battery of benchmarks.

Questions to ask include:

- What happens when a GPU fails?
- How quickly can failed GPUs be replaced?
- How many spare nodes must be reserved?
- What is the SLA for replacement?
- What network topology is actually provided?
- Are collectives benchmarked at realistic node counts?
- What storage backend is mounted by default?
- What are the read and write rates for small and large files?
- Which environment variables are required for full performance?

- Are features such as SHARP or NVLink Switch enabled?

The speaker’s stance is not that every provider is malicious, but that providers often optimize for selling capacity while users need engineering guarantees. The user must measure.

Marketing Versus Reality

A vendor specification is a reference point, not a contract with physics.

For each component:

advertised number $\geq$ microbenchmark number $\geq$ isolated workload number $\geq$ end-to-end workload number.

The gap can be caused by:

- hidden assumptions in the benchmark;
- bidirectional versus unidirectional reporting;
- decimal versus binary units;
- boost clock versus sustained clock;
- sparse versus dense FLOP reporting;
- payload sizes not matching the real workload;
- software framework overheads;
- caching effects;
- cluster topology and congestion.

Hardware-Level Explanation

Accelerator Compute and Clock Behavior

The theoretical FLOP rate depends on clock speed and compute resources:

$$P_{\text{theoretical}} = f_{\text{clock}} \cdot N_{\text{units}} \cdot O_{\text{per unit per cycle}}.$$

Here:

- f_{clock} is the clock frequency;
- N_{units} is the number of compute units;
- $O_{\text{per unit per cycle}}$ is the number of operations each unit can issue per cycle.

The problem is that f_{clock} is not constant. Under thermal or power constraints, the GPU may throttle:

$$f_{\text{sustained}} < f_{\text{boost}}.$$

A short benchmark may observe boost-clock behavior. A real training workload with continuous GEMMs, collectives, optimizer steps, and checkpointing may force lower sustained clocks.

Thermal Throttling and Slow Outlier GPUs

If a GPU overheats or has weaker cooling, it may reduce clock frequency. The step time of synchronous training is controlled by the slowest participant:

$$T_{\text{step}} = \max(T_1, T_2, \dots, T_G).$$

Suppose $G - 1$ GPUs complete a step in 1.0 seconds, but one GPU takes 1.4 seconds. The whole job runs at 1.4 seconds per step:

$$\text{slowdown} = \frac{1.4}{1.0} = 1.4.$$

This creates a 40% slowdown for the entire synchronous job.

Memory Bandwidth and the Declining Efficiency of Newer GPUs

The speaker observes that advertised compute grows faster than memory bandwidth. A simplified generational model is:

$$P_{\text{compute,new}} \approx \alpha P_{\text{compute,old}}, \quad B_{\text{memory,new}} \approx \beta B_{\text{memory,old}},$$

with

$$\alpha > \beta.$$

If compute improves by 2x or 3x while memory bandwidth improves much less, then more operations become limited by feeding data to the compute units.

For an operation with arithmetic intensity

$$I = \frac{\text{FLOPs}}{\text{bytes moved}},$$

the roofline bound is

$$P_{\text{achievable}} = \min(P_{\text{peak}}, B_{\text{memory}} \cdot I).$$

Even if P_{peak} increases dramatically, achievable performance remains limited by $B_{\text{memory}} \cdot I$ when arithmetic intensity is not high enough.

Why Fused Kernels Can Exceed Standalone GEMM Intuition

The speaker notes that standalone matrix multiplication benchmarks include the overhead of moving data from HBM into compute. Fused kernels may achieve better effective performance by reducing memory traffic.

For two separate operations,

$$Y = f(X), \quad Z = g(Y),$$

a naive implementation may write Y to HBM and read it back:

$$\text{bytes}_{\text{naive}} = \text{read}(X) + \text{write}(Y) + \text{read}(Y) + \text{write}(Z).$$

A fused implementation can keep Y in registers or shared memory:

$$\text{bytes}_{\text{fused}} = \text{read}(X) + \text{write}(Z).$$

The reduced memory traffic increases arithmetic intensity:

$$I_{\text{fused}} > I_{\text{naive}}.$$

This is why kernel fusion can matter even when a single isolated GEMM benchmark looks memory-limited.

Intra-Node Networking

Intra-node communication occurs among GPUs in the same server. Technologies include:

- PCIe;
- NVLink;
- NVSwitch;
- AMD Infinity Fabric;
- other vendor-specific local interconnects.

Intra-node communication is crucial for tensor parallelism, pipeline parallelism inside the node, and fast local collectives.

Tensor parallelism is especially communication-heavy because intermediate activations or partial results must be exchanged frequently. The lecture strongly implies the rule:

tensor parallelism $\rightarrow$ keep inside a fast local fabric whenever possible.

Inter-Node Networking

Inter-node communication connects different servers. It is required when a model or training job scales beyond a single node.

The hierarchy matters:

$$B_{\text{intra-node}} > B_{\text{inter-node}}$$

in many practical systems, even when the gap has narrowed in recent hardware.

A parallelism strategy must map communication-heavy dimensions onto the fastest available network. A poor mapping can expose slow inter-node communication and reduce MFU.

Communication Overlap

During backpropagation, gradients become available layer by layer. Instead of waiting until the full backward pass finishes, a framework can start reducing gradients as soon as a bucket is ready.

For layer l , let

$$T_{\text{backward},l}$$

be the compute time and

$$T_{\text{reduce},l}$$

be the communication time. If communication fits under remaining backward compute, it is hidden:

$$T_{\text{reduce},l} \leq \sum_{j < l} T_{\text{backward},j}.$$

If not, exposed communication appears:

$$T_{\text{exposed}} = \max(0, T_{\text{communication}} - T_{\text{overlap window}}).$$

The iteration time becomes approximately:

$$T_{\text{iteration}} = T_{\text{compute}} + T_{\text{exposed}} + T_{\text{data}} + T_{\text{other}}.$$

Blocking Communication

Some communication cannot be hidden. In pipeline parallelism, a stage may require activations from a previous stage before it can compute:

$$h_{l+1} = f_l(h_l).$$

If h_l has not arrived, the next stage cannot proceed. Therefore:

$$T_{\text{stage}} = T_{\text{receive activation}} + T_{\text{compute}} + T_{\text{send activation}}.$$

Fast networking is essential when communication is on the critical path.

Storage as a Hardware Bottleneck

Storage matters for at least three reasons:

1. feeding training data fast enough;
2. loading checkpoints during startup or restart;
3. writing checkpoints frequently enough to survive failures.

A dataloader that cannot prepare data before the GPU finishes the previous step creates idle accelerator time:

$$T_{\text{idle}} = \max(0, T_{\text{data}} - T_{\text{previous compute}}).$$

Checkpointing can also become a major cost. If a checkpoint takes T_{ckpt} minutes and is written every H hours, then checkpoint overhead per day is

$$T_{\text{overhead per day}} = \frac{24}{H} T_{\text{ckpt}}.$$

For example, if $T_{\text{ckpt}} = 5$ minutes and $H = 3$, then

$$T_{\text{overhead per day}} = \frac{24}{3} \cdot 5 = 40 \text{ minutes}.$$

This is not negligible at large GPU counts.

Mathematical Reasoning

Realistic Maximum Achievable FLOPs

The speaker proposes replacing vendor peak FLOPs with measured maximum achievable FLOPs for the target hardware.

Let

$$P_{\text{vendor}}$$

be the advertised peak and

$$P_{\text{measured}}(M, N, K, d)$$

be measured GEMM throughput for matrix shape (M, N, K) and data type d . Then define

$$P_{\text{MAF}} = \max_{(M, N, K) \in \mathcal{S}} P_{\text{measured}}(M, N, K, d),$$

where $\mathcal{S}$ is a set of benchmarked shapes.

For a specific workload with shape set $\mathcal{W}$, an even better denominator is:

$$P_{\text{workload max}} = \text{aggregate measured throughput over shapes in } \mathcal{W}.$$

Then:

$$\text{Measured-utilization ratio} = \frac{P_{\text{actual workload}}}{P_{\text{workload max}}}.$$

This gives a more honest optimization target.

Dense Versus Sparse Performance Reporting

If sparse acceleration assumes many zeros, then the reported sparse peak may be roughly:

$$P_{\text{sparse}} \approx 2P_{\text{dense}}.$$

If a workload is dense but one uses sparse peak in the denominator, the apparent utilization is artificially low:

$$\text{MFU}_{\text{wrong}} = \frac{P_{\text{actual}}}{P_{\text{sparse}}} \approx \frac{1}{2} \frac{P_{\text{actual}}}{P_{\text{dense}}}.$$

Thus, a workload that is actually using 80% of dense peak may appear to use only 40% of sparse-advertised peak.

Mixed Precision Complicates MFU

Training is often not purely BF16, FP16, FP8, or FP32. Some operations may run in lower precision, while reductions, normalization, optimizer states, or accumulations may use higher precision.

If a workload has components $i = 1, \dots, n$, with operation counts F_i and achievable hardware rates P_i , then ideal compute time is:

$$T_{\text{ideal}} = \sum_{i=1}^n \frac{F_i}{P_i}.$$

A single denominator P_{BF16} can misrepresent a mixed-precision workload:

$$P_{\text{effective}} \neq P_{\text{BF16}} \quad \text{when operations use multiple dtypes.}$$

Network Bandwidth Units

Networking numbers are often confusing because of:

- bits versus bytes;
- decimal base 10 versus binary base 2;
- unidirectional versus bidirectional bandwidth.

Conversion from bits per second to bytes per second is:

$$B_{\text{bytes/s}} = \frac{B_{\text{bits/s}}}{8}.$$

Decimal giga and binary gibi differ:

$$1 \text{ GB} = 10^9 \text{ bytes}, \quad 1 \text{ GiB} = 2^{30} \text{ bytes}.$$

The ratio is:

$$\frac{1 \text{ GiB}}{1 \text{ GB}} = \frac{2^{30}}{10^9} \approx 1.074.$$

Bidirectional bandwidth can be approximately twice unidirectional bandwidth:

$$B_{\text{bidirectional}} \approx 2B_{\text{unidirectional}}.$$

Benchmarks such as collective communication tests often report unidirectional or normalized bus bandwidth, so comparisons must use matching definitions.

Payload Size and Effective Bandwidth

Network throughput depends on message size. Let:

S = payload size in bytes,

L = latency overhead,

B = asymptotic bandwidth.

A simplified communication time model is:

$$T_{\text{comm}}(S) = L + \frac{S}{B}.$$

Effective bandwidth is:

$$B_{\text{effective}}(S) = \frac{S}{L + S/B}.$$

For small S , latency dominates:

$$B_{\text{effective}}(S) \approx \frac{S}{L}.$$

For large S , bandwidth dominates:

$$B_{\text{effective}}(S) \approx B.$$

This explains why the same network can look slow for small buckets and fast for very large transfers.

Communication Bucket Sizing

Frameworks such as DDP and DeepSpeed often reduce gradients in buckets. Let:

S_{bucket} = bucket size,

$$T_{\text{bucket}} = L + \frac{S_{\text{bucket}}}{B}.$$

Larger buckets improve bandwidth efficiency but may reduce overlap opportunities. Smaller buckets start earlier but pay more latency overhead.

The tuning problem is:

$$\min_{S_{\text{bucket}}} T_{\text{iteration}}(S_{\text{bucket}}),$$

where

$$T_{\text{iteration}}(S_{\text{bucket}}) = T_{\text{compute}} + T_{\text{exposed comm}}(S_{\text{bucket}}) + T_{\text{other}}.$$

There is no universally optimal bucket size. It depends on model architecture, network, framework, and layer gradient sizes.

Storage Throughput

For storage, two common metrics are:

$$\text{throughput} = \frac{\text{bytes read or written}}{\text{time}},$$

and

$$\text{IOPS} = \frac{\text{I/O operations}}{\text{time}}.$$

Small files stress metadata and IOPS. Large checkpoint files stress streaming bandwidth.

If a package import opens N small files and each file operation has metadata latency L_{meta} , then rough import time can contain:

$$T_{\text{import}} \approx NL_{\text{meta}} + \frac{\text{total bytes}}{B_{\text{storage}}}.$$

This is why importing a library with many small files can be a surprisingly effective file-system benchmark.

Code Walkthrough

Logging Environment Versions for Reproducibility

A serious training or benchmarking run should record the full software environment. The exact package versions, CUDA version, git commit hashes, and environment variables can change performance and correctness.

```

import json
import os
import platform
import subprocess
import sys
from pathlib import Path

def run(cmd):
    return subprocess.check_output(
        cmd,
        shell=True,
        text=True,
        stderr=subprocess.STDOUT
    ).strip()

def safe_run(cmd):
    try:
        return run(cmd)
    except Exception as exc:
        return f"FAILED: {exc}"

metadata = {
    "python": sys.version,
    "platform": platform.platform(),
    "executable": sys.executable,
    "pip_freeze": safe_run("python -m pip freeze"),
    "nvidia_smi": safe_run("nvidia-smi"),
    "cuda_compiler": safe_run("nvcc --version"),
    "environment": dict(os.environ),
    "git_commit": safe_run("git rev-parse HEAD"),
    "git_status": safe_run("git status --short"),
}

Path("run_metadata.json").write_text(
    json.dumps(metadata, indent=2, sort_keys=True)
)

```

Major ideas in this snippet:

- `pip_freeze` records Python package versions.
- `nvidia-smi` records GPU model, driver, power, and utilization information.
- `nvcc --version` records CUDA compiler details.
- `os.environ` records performance-relevant environment variables.
- `git rev-parse HEAD` records the exact source revision.

This matters because performance can shift when a single environment variable or package version changes. For distributed training, environment variables can enable or disable major communication features.

Simple GEMM Throughput Benchmark

The following benchmark captures the principle behind measuring maximum achievable FLOPs for actual matrix shapes.

```
import time
import torch

def benchmark_gemm(M, N, K, dtype=torch.bfloat16, iters=100):
    device = "cuda"
    A = torch.randn((M, K), device=device, dtype=dtype)
    B = torch.randn((K, N), device=device, dtype=dtype)

    for _ in range(10):
        C = A @ B
        torch.cuda.synchronize()

    start = time.perf_counter()
    for _ in range(iters):
        C = A @ B
        torch.cuda.synchronize()
    end = time.perf_counter()

    elapsed = (end - start) / iters
    flops = 2 * M * N * K
    tflops = flops / elapsed / 1e12
    return elapsed, tflops

shapes = [
    (1024, 1024, 1024),
    (2048, 2048, 2048),
    (4096, 4096, 4096),
    (8192, 8192, 8192),
]

for M, N, K in shapes:
    elapsed, tflops = benchmark_gemm(M, N, K)
    print((M, N, K), elapsed, tflops)
```

Important hardware behavior:

- `A @ B` dispatches a matrix multiplication kernel, usually through a vendor library.

- Larger matrices often achieve higher utilization because launch overhead and fixed costs are amortized.
- The result depends on dtype, shape alignment, GPU clock behavior, memory bandwidth, and library version.
- The warmup loop avoids measuring one-time initialization costs.
- `torch.cuda.synchronize()` is required because CUDA kernels launch asynchronously.

The measured FLOP rate is:

$$P_{\text{measured}} = \frac{2MNK}{T_{\text{elapsed}}}.$$

This benchmark should be run on the exact hardware and software stack used for the real workload.

Measuring Sustained Performance Rather Than Boost Performance

A short benchmark can overestimate performance if it only observes boost clocks. A sustained benchmark repeats work long enough to expose thermal throttling.

```
import time
import torch

def sustained_gemm(M=8192, N=8192, K=8192, seconds=120):
    device = "cuda"
    A = torch.randn((M, K), device=device, dtype=torch.bfloat16)
    B = torch.randn((K, N), device=device, dtype=torch.bfloat16)

    torch.cuda.synchronize()
    start = time.perf_counter()
    count = 0

    while time.perf_counter() - start < seconds:
        C = A @ B
        count += 1

    torch.cuda.synchronize()
    end = time.perf_counter()

    total_flops = count * 2 * M * N * K
    return total_flops / (end - start) / 1e12

print("Sustained TFLOP/s:", sustained_gemm())
```

This benchmark is closer to training because it applies continuous load. If the first 10 seconds look fast but the 120-second average drops, thermal or power behavior is affecting sustained training.

Poor Man's Storage Benchmark: Importing PyTorch

The speaker mentions a simple practical benchmark:

```
python -c "import torch".
```

The reason it works is that importing PyTorch opens many small files. Small-file metadata performance can dominate on weak shared file systems.

```
# Run from a shell:
time python -c "import torch"
```

To avoid fooling oneself with file-system cache effects, one must understand that repeated runs may be faster because data is cached. On Linux, cache dropping requires root privileges and should be done carefully on non-production systems.

```
# Dangerous on shared systems. Use only when appropriate:
sync
echo 3 | sudo tee /proc/sys/vm/drop_caches
time python -c "import torch"
```

The benchmark interpretation is:

$$T_{\text{cold import}} \gg T_{\text{warm import}}$$

when the file system has high metadata latency but caching hides it on repeated runs.

Representative FIO-Style Storage Benchmark

The speaker recommends benchmarking both small-file and large-file behavior. A conceptual script can generate random reads and writes with different block sizes.

```
# Example commands to adapt for a target file system:
fio --name=small_read \
    --directory=/path/to/test \
    --rw=randread \
    --bs=16k \
    --size=4G \
    --numjobs=8 \
    --time_based \
    --runtime=60 \
```

```

--group_reporting

fio --name=large_write \
    --directory=/path/to/test \
    --rw=write \
    --bs=1G \
    --size=64G \
    --numjobs=1 \
    --time_based \
    --runtime=60 \
    --group_reporting

```

Small block sizes approximate Python package imports and many tiny dataset files. Large block sizes approximate checkpoint read and write behavior.

Gradient Communication Bucket Model

A minimal communication model can be written in Python to reason about exposed communication.

```

def effective_bandwidth(payload_bytes, latency_s, bandwidth_bytes_s):
    time_s = latency_s + payload_bytes / bandwidth_bytes_s
    return payload_bytes / time_s

def communication_time(payload_bytes, latency_s, bandwidth_bytes_s):
    return latency_s + payload_bytes / bandwidth_bytes_s

def exposed_comm_time(payload_bytes, latency_s, bandwidth_bytes_s, overlap_window_s):
    t_comm = communication_time(payload_bytes, latency_s, bandwidth_bytes_s)
    return max(0.0, t_comm - overlap_window_s)

payload_sizes = [16 << 20, 256 << 20, 2 << 30]
latency_s = 20e-6
bandwidth_bytes_s = 200 * (10 ** 9)
overlap_window_s = 0.004

for payload in payload_sizes:
    exposed = exposed_comm_time(
        payload,
        latency_s,
        bandwidth_bytes_s,
        overlap_window_s
    )
    print(payload, "exposed seconds:", exposed)

```

This is not a substitute for real benchmarking. It is a thinking tool. It shows why a larger bucket may improve bandwidth but may also change whether

communication fits under available compute.

Representative Distributed Training Timing Instrumentation

The lecture repeatedly emphasizes measuring every part of the iteration. A simple timing pattern is:

```
import time
import torch

def sync_time():
    torch.cuda.synchronize()
    return time.perf_counter()

def train_step(model, batch, optimizer):
    t0 = sync_time()

    inputs, targets = batch
    inputs = inputs.cuda(non_blocking=True)
    targets = targets.cuda(non_blocking=True)

    t1 = sync_time()

    outputs = model(inputs)
    loss = loss_fn(outputs, targets)

    t2 = sync_time()

    loss.backward()

    t3 = sync_time()

    optimizer.step()
    optimizer.zero_grad(set_to_none=True)

    t4 = sync_time()

    return {
        "host_to_device": t1 - t0,
        "forward": t2 - t1,
        "backward": t3 - t2,
        "optimizer": t4 - t3,
        "total": t4 - t0,
    }
```

Important details:

- CUDA operations are asynchronous, so synchronization is needed for accurate timing.
- This simple version may perturb performance; production profiling should use more careful instrumentation.
- Breaking time into components reveals whether the bottleneck is data movement, forward compute, backward compute, optimizer work, or communication.

Performance Intuition

Do Not Optimize Against Fictional Peak

If the advertised peak is P_{vendor} , but the best measured standalone performance is P_{measured} , then chasing P_{vendor} may be futile.

The correct engineering process is:

benchmark hardware $\rightarrow$ define realistic 100% $\rightarrow$ measure workload $\rightarrow$ optimize gap $\rightarrow$ stop when returns diminish

If a workload reaches 85% of measured achievable performance, it may already be excellent even if it is only 50% of a marketing number.

The Diminishing Returns Decision

Let p be current utilization relative to realistic maximum, and let Δp be expected improvement from optimization. The value of optimizing is roughly:

$$\text{value} = \Delta p \cdot C_{\text{run}},$$

where C_{run} is total run cost.

If a pretraining run costs millions of dollars and lasts months, a 1% improvement may be worth substantial engineering effort. If a fine-tuning run lasts a few days, the same 1% may not matter.

This is a central practical distinction:

pretraining $\rightarrow$ small utilization gains can be huge,

short fine-tuning $\rightarrow$ engineering time may dominate savings.

Communication Can Be Hidden Only If It Fits Under Compute

A common misconception is that asynchronous communication automatically eliminates communication cost. In reality, communication is hidden only if:

$$T_{\text{comm}} \leq T_{\text{available compute overlap}}.$$

Otherwise exposed communication remains:

$$T_{\text{exposed}} = T_{\text{comm}} - T_{\text{available compute overlap}}.$$

This is why network quality, bucket size, and layer ordering matter.

Message Size Matters

A benchmark that sends 16 GiB messages may report excellent bandwidth. A real workload may send 100 MiB or 256 MiB gradient buckets. Because of latency and protocol overhead, the smaller payload may achieve much lower effective bandwidth.

Therefore:

$$B_{\text{effective}}(100 \text{ MiB}) \neq B_{\text{effective}}(16 \text{ GiB}).$$

One should benchmark the actual message sizes used by the framework.

Parallelism Strategy Must Respect Network Topology

A rough rule is:

$$\text{most frequent communication} \rightarrow \text{fastest network tier}.$$

Thus:

- tensor parallelism usually belongs inside a node;
- data parallel gradient reductions can often span nodes;
- pipeline stages should be placed carefully because activation transfers can be blocking;
- FSDP or ZeRO-style sharding can be poor on weak networks because parameters and gradients are frequently prefetched or reduced.

Data Loading Is Part of GPU Utilization

A GPU can compute only after data arrives. If dataloader workers perform expensive preprocessing on the critical path, accelerator utilization falls.

The desired pattern is:

data preparation for step $t + 1$ overlaps with GPU compute for step t .

If this overlap fails:

$$T_{\text{iteration}} = T_{\text{data}} + T_{\text{compute}} + \dots$$

instead of:

$$T_{\text{iteration}} = \max(T_{\text{data}}, T_{\text{compute}}) + \dots.$$

This is especially important for computer vision and video workloads, where decoding and augmentation can be heavy.

Checkpointing Is a Reliability-Performance Trade-Off

Frequent checkpoints reduce lost work after failure:

$$T_{\text{lost}} \leq T_{\text{checkpoint interval}}.$$

But each checkpoint costs time:

$$T_{\text{checkpoint overhead}} = N_{\text{checkpoints}} T_{\text{write}}.$$

At large GPU counts, failures become more likely. If the probability of individual GPU failure over a period is p , then the probability that at least one of G GPUs fails is:

$$P_{\text{any failure}} = 1 - (1 - p)^G.$$

For large G , this rises quickly. Therefore, checkpointing strategy is not optional at scale.

Framework Defaults Are Starting Points, Not Truth

Default settings in DDP, FSDP, DeepSpeed, NCCL, or cloud environments may not be optimal. Environment variables can dramatically change behavior.

Examples of sensitive settings include:

- communication algorithm choices;
- enabling or disabling network offload features;
- CUDA launch behavior;
- NCCL topology detection;
- library preloading;
- shared-object paths;
- bucket sizes.

The practical lesson is to compare full environments when reproducing performance:

same code + different environment $\Rightarrow$ different performance.

Common Misconceptions

Misconception: A GPU Specification Predicts Training Speed

A GPU spec gives only a theoretical upper bound under specific assumptions. Real training speed depends on shape, dtype, thermal behavior, memory bandwidth, networking, storage, framework overhead, and failures.

Correct view:

training speed = end-to-end measured property.

Misconception: MFU Is Always an Objective Measure

MFU depends on the denominator. If the denominator is a sparse marketing peak or a pure BF16 peak for a mixed-precision workload, the result can be misleading.

Correct view:

MFU is useful only when the denominator is meaningful.

Misconception: Newer Hardware Is Always More Efficient

Newer hardware may have higher absolute throughput but lower percentage efficiency relative to advertised peak because compute growth can outpace memory bandwidth growth.

Correct view:

absolute performance $\neq$ efficiency relative to advertised peak.

Misconception: Networking Is Only a Distributed Systems Detail

Networking can directly determine GPU utilization. Exposed communication stalls compute. Blocking pipeline transfers can dominate iteration time.

Correct view:

network topology $\in$ core ML performance model.

Misconception: Storage Is Secondary

Slow storage can starve dataloaders, delay startup, slow checkpoint writes, and make development painful. Importing a package can become slow if the file system handles many small files poorly.

Correct view:

storage performance $\in$ training throughput and reliability.

Misconception: If Loss Decreases, the Training Code Is Correct

The speaker gives an example from BLOOM training where a synchronization issue could have produced a model tied to a particular distributed setup. SGD can sometimes adapt around bugs, so decreasing loss does not prove correctness.

Correct view:

loss improvement $\nRightarrow$ implementation correctness.

Misconception: Social Media Is a Durable Technical Portfolio

Short-lived social media posts disappear quickly from attention. Durable artifacts such as books, repositories, technical notes, and living documents accumulate

value over time.

Correct view:

long-lived artifacts > ephemeral attention.

Practical Takeaways

Engineering Habits

1. Write down every nontrivial failure and fix.
2. Record package versions, git hashes, CUDA versions, driver versions, and environment variables.
3. Benchmark the actual hardware before committing to large runs.
4. Benchmark the actual matrix shapes, dtypes, message sizes, and storage access patterns.
5. Treat cloud-provider claims as hypotheses to validate.
6. Build relationships with engineers who understand PyTorch, CUDA, NCCL, storage, scheduling, and networking.
7. Create long-lived public or private technical artifacts.

Hardware Evaluation Checklist

For a new cluster or cloud provider, measure:

- sustained GEMM throughput for workload-specific shapes;
- thermal behavior over long runs;
- per-GPU variance and outliers;
- intra-node all-reduce bandwidth;
- inter-node all-reduce bandwidth at realistic node counts;
- point-to-point latency and bandwidth;
- storage small-file IOPS;
- storage large-file read and write bandwidth;
- checkpoint save and load time;
- dataloader throughput;
- failure replacement workflow.

Distributed Training Checklist

Before scaling a training job, answer:

- Which communication operations are blocking?
- Which communication operations can overlap with compute?
- What are the gradient bucket sizes?
- What payload sizes occur in tensor parallelism?
- Does tensor parallelism cross nodes?
- Does FSDP or ZeRO require frequent parameter prefetch over a slow network?
- How often are checkpoints written?
- How much work is lost after a failure?
- Are all GPUs performing similarly?

Cloud Contract Checklist

Before accepting a cloud deal, ask:

- Can we benchmark representative nodes before signing?
- Are benchmark results part of acceptance criteria?
- What happens when measured performance is far below claims?
- What is the replacement process for bad GPUs?
- How many spare nodes are available?
- Who are the engineers we can contact during incidents?
- What storage system is included by default?
- What network topology is actually allocated?
- Which performance features are enabled or disabled?

Project or Experiment Ideas

Experiment 1: Vendor Peak Versus Measured GEMM Peak

Benchmark several GEMM shapes on one GPU:

$(M, N, K) \in \{(1024, 1024, 1024), (2048, 2048, 2048), (4096, 4096, 4096), (8192, 8192, 8192)\}$.

Measure:

$$P_{\text{measured}} = \frac{2MNK}{T}.$$

Compare against the advertised dense peak, not sparse peak. Plot measured throughput versus matrix size.

Experiment 2: Sustained Clock Behavior

Run the same GEMM benchmark for 10, 60, and 300 seconds. Record:

$$P_{10}, \quad P_{60}, \quad P_{300}.$$

If:

$$P_{10} > P_{300},$$

then boost behavior or thermal throttling may be present.

Experiment 3: Dataloader Starvation

Create two dataloaders:

- one with preprocessed tensors;
- one with expensive CPU-side transforms.

Measure:

$$T_{\text{data}}, \quad T_{\text{compute}}, \quad T_{\text{idle}}.$$

Increase `num_workers` and observe whether GPU utilization improves.

Experiment 4: Storage Small Files Versus Large Files

Run two storage tests:

- many 16 KiB random reads;
- large 1 GiB sequential writes.

Compare IOPS and throughput. Then run:

```
python -c "import torch"
```

on the same file system to see whether package import time correlates with small-file performance.

Experiment 5: Communication Payload Size Sweep

Using NCCL tests or a PyTorch distributed benchmark, sweep payload sizes:

$$S \in \{32 \text{ KiB}, 1 \text{ MiB}, 16 \text{ MiB}, 256 \text{ MiB}, 2 \text{ GiB}, 16 \text{ GiB}\}.$$

Measure effective bandwidth:

$$B_{\text{effective}}(S) = \frac{S}{T(S)}.$$

Identify the payload size where bandwidth approaches its plateau.

Experiment 6: Bucket Size Tuning

Train a small distributed model and vary gradient bucket size:

$$S_{\text{bucket}} \in \{25 \text{ MiB}, 100 \text{ MiB}, 256 \text{ MiB}, 1 \text{ GiB}\}.$$

Measure:

$$T_{\text{step}}, \quad \text{MFU}, \quad T_{\text{exposed communication}}.$$

The goal is to observe the trade-off between latency amortization and overlap.

Experiment 7: Environment Diff Debugging

Run the same benchmark on two machines or two shells. Save:

- `pip freeze`;
- `nvidia-smi`;
- git commit hash;
- all environment variables;
- benchmark output.

When performance differs, diff the logs. This reproduces the speaker’s emphasis on environment variables and reproducibility.

Visual Concept

Draw the ML training system as a pipeline. On the left, show storage feeding dataloader workers on CPU. Then show host-to-device transfer into GPUs. Inside the GPU box, draw compute, HBM, SMs, and tensor cores. Between GPUs, draw intra-node links. Between nodes, draw a slower inter-node network. Above the pipeline, annotate possible bottlenecks: data loading, compute, communication, checkpointing, logging, and failures.

Final Mental Model

The lecture's final mental model is:

A resilient ML engineer does not trust isolated claims, isolated benchmarks, or isolated tools. They build a habit of measuring the actual system, recording what changed, understanding where time goes, and cultivating people and artifacts that survive the next framework, model, GPU, or cloud-provider shift.

In formula form:

resilient ML engineering = measurement+debugging discipline+systems thinking+relationships+durable know

A model training run is not only a sequence of tensor operations. It is an end-to-end engineered system whose throughput is constrained by the slowest relevant component:

useful throughput = $\min(\text{accelerator compute, memory bandwidth, network collectives, storage throughput, so}$

The practical engineering stance is therefore:

1. Measure the territory, not the map.
2. Define a realistic 100% from your own benchmarks.
3. Optimize only after identifying the true bottleneck.
4. Stop when additional optimization no longer pays for itself.
5. Write everything down so the next problem becomes easier.